

Autonomous Bitcoin Trading

Design and Evaluation of an Autonomous Bitcoin Trading Algorithm

High-School Thesis (Maturaarbeit)

Empirical Investigation

Author: [JuriVX]

Class: [Redacted]

Supervisor: [Redacted]

School: [Redacted]

Submission Date: September 7, 2025

Abstract

This thesis describes the development of an autonomous Bitcoin trading algorithm, based on the observation that human emotions often cause irrational decisions in financial markets. The research combines technical analysis of market data with algorithmic trading, following a quantitative approach that includes data collection, strategy development in Python, backtesting, and live testing.

This work addresses the central research question of whether an individual can design an autonomous trading algorithm that achieves positive returns or outperforms a simple buy-and-hold strategy in Bitcoin over a live testing period.

Because no service provided the required historical order book time series, a custom Python script was developed to gather data from multiple APIs, store it in a PostgreSQL database, and compute order book delta metrics. A custom backtesting framework was implemented to test rule-based trading strategies under changing parameters and sample sizes.

After identifying a promising strategy, the algorithm was deployed on a Linux-based server and ran from August 14 to September 5, 2025. During this live period, the strategy executed 17 trades, achieving a Sharpe ratio of -0.007 but outperforming Bitcoin by $+5.59\%$, as the benchmark Sharpe ratio from Bitcoin was -2.35 .

These results suggest that short-term inefficiencies can sometimes be exploited, but the small sample size limits the strength of the evidence. The findings should therefore be interpreted as an indication of potential rather than proof of a persistent trading edge.

Foreword

My interest in Bitcoin began in early 2022 after reading *The Bitcoin Standard by Saifedean Ammous* (Ammous, 2018). The book explores the history of money and argues that Bitcoin is a revolutionary form of hard money ¹ superior to traditional currencies. The book sparked an interest in me, and I was drawn into the world of cryptocurrencies. As a result, I spent more and more time watching charts and losing my mind over them. The idea of making profit from these seemingly random price movements fascinated me. Trading profitably is straightforward: sell high and buy low. The practical implementation, however, is filled with uncertainty and a fight against your own instincts and emotions. In addition to getting into crypto I also tried to learn the programming language JavaScript since I wanted to make money on the side without having to leave my house. The first time I heard about automated trading algorithms I thought it was one of the hardest things to pursue. The combination of market understanding and programming made me admire people who wrote computer programs in the finance space. It felt impossible but the more I tried the closer I got to something that was similar to a trading algorithm. With my maturation thesis approaching I was well aligned with the idea of writing a thesis about the process of programming a trading algorithm.

I had no idea what I was getting myself into, and if I had known before, I would probably have never started. I spent more time than I intended to and completely lost myself during the summer vacation. I totally forgot about the writing part and spent a lot of time just coding. This project grew much bigger than I initially thought, but I am proud to present both outcome and the learnings I gained from it.

.

¹A form of money that maintains its value over time, offering stability and reliability

Glossary

Algorithm A set of rules or instructions designed to solve a problem or complete a task, in this case, for automated trading.

API (Application Programming Interface) A set of protocols and tools that allow different software applications to communicate with each other.

Backtesting The process of testing a trading strategy using historical data to evaluate its potential performance.

Bitcoin A decentralized digital cryptocurrency created in 2009 by an anonymous entity known as Satoshi Nakamoto.

EMH (Efficient Market Hypothesis) A theory stating that asset prices reflect all available information, making it difficult to consistently outperform the market.

IQR (Interquartile Range) A measure of statistical dispersion, calculated as the difference between the third quartile (Q3) and first quartile (Q1).

Linear Regression A statistical method used to model the relationship between variables by fitting a straight line through data points, commonly used to calculate trendline slopes.

Order book A real-time list of buy and sell orders for a specific asset, showing quantities traders are willing to trade at different price levels.

Order book Delta The difference between the total volume of bid orders and ask orders within a certain price range from the current market price.

PaaS (Platform as a Service) A cloud computing service that provides a platform for developing, running, and managing applications.

Sharpe Ratio A measure of risk-adjusted return, calculated as the ratio of excess return to standard deviation of returns.

Slippage The difference between the expected price of a trade and the actual executed price due to market conditions.

Systematic Strategy A trading approach based on predefined rules and mathematical models rather than discretionary decision-making.

Volatility A measure of how much and how quickly the price of a trading instrument fluctuates over time.

Contents

1	Introduction	5
2	Theoretical Background	6
2.1	What is trading?	6
2.2	What is Bitcoin?	7
2.3	Efficient markets	9
2.4	Development of a Systematic Strategy	10
2.5	Order book	11
3	Methodology	12
3.1	Project Overview	12
3.2	Services used	13
3.3	Data collection	14
3.4	Defining goals	17
3.5	Robustness Check via Resampled Strategy Variants	17
3.6	Strategy example	20
3.7	My strategy	22
4	Results	29
4.1	Backtest results	29
4.2	Backtest results in comparison to live trading	31
4.3	Strategy evaluation	32
5	Discussion	33
5.1	Interpretation of results	33
5.2	Challenges	33
5.3	Limitations	34
5.4	Practical insights	34
5.5	Future work	34
5.6	Final thoughts	35
A	Appendix	36
A.1	List of figures	36
A.2	Code sources	37
A.3	List of tables	37
A.4	Published Github repositories	37

1 Introduction

Making money in the capital markets is a complex activity that requires emotional control and discipline. Most individuals fail to make money on shorter time frames. Since computers do not suffer from psychological biases, automating the trading execution process offers the potential for more consistent decision making. This thesis documents the development of an autonomous trading algorithm designed to generate consistent profits. This work does not aim to outperform professional trading firms, but rather to demonstrate the challenges of building a fully automated trading algorithm. To that end, my thesis will cover the entire development cycle of an algorithmic trading strategy. This will include finding a trading strategy, backtesting the trading strategy, applying the strategy to a live market, and evaluating the results.

The central research question of this thesis is therefore: Is it possible for an individual researcher to develop an autonomous trading algorithm that generates positive returns or outperforms a buy and hold strategy in Bitcoin over a live testing period? This question will serve as the guiding framework for the project, from initial design to final evaluation.

Finding a strategy can be done in multiple ways just by watching markets over longer time periods to get ideas, reading books about how other people were able to develop strategies and copy logic from them. This project attempts to identify a strategy by analyzing the order book delta (1) and identifying trends in Bitcoin price as it is non-stationary.²

Live trading is the process of executing a trading strategy on a live market. During this period the strategy will be put to the test. The strategy will be automated using Python. A major difference is that two different Python codebases were used for the backtesting process and the live trading algorithm.

The project requires know-how in three key areas: statistics, programming and understanding how the cryptocurrency market operates. Statistical rules are applied to evaluate the strategy's potential for profitable automated execution. Programming is just needed to make the process of testing or applying my ideas easy on large datasets since testing must be done on time series. The algorithm will be written in Python and will run 24 hours a day seven days a week.

By the end of the project, I aim to deliver the following components:

- Backtest results
- Trading strategy
- Trading algorithm program
- Live execution results

²Non-stationary means not being still or fixed in one place; moving or changing rather than remaining constant.

2 Theoretical Background

Automating a trading strategy comes with many different challenges which are not obvious at first. In this section I will explain the basics needed to understand the trading market, algorithmic trading and challenges of creating a trading algorithm.

2.1 What is trading?

Trading means buying and selling financial assets like stocks, cryptocurrencies, or commodities, usually with the goal of making a profit. Traders try to buy at a low price and sell at a higher one, or profit from falling prices by selling high and buying back lower. Prices in financial markets change constantly, which creates opportunities and risks. There are many different types of trading, depending on the chosen timeframe, strategy, and how trades are managed. Losing is part of every strategy. The goal is to lose less money as the winning amount. As trading is often seen as an easy way to make money it is important to understand that it is not. It is a skill that requires a lot of time and effort to acquire. Regardless of which trading strategy one chooses, there is no easy or fast path to success in trading.

Algorithmic Trading

This thesis investigates the potential of algorithmic trading, also known as Algo-trading or automated trading. This is a type of trading which uses computer programs to execute trades in financial markets based on predefined rules or mathematical models. Inside of algorithmic trading there are again many different types. Some examples are: statistical arbitrage, trend following, and high-frequency trading. In terms of logic all these are not related at all and strategies are often very complex in these areas.

The strategy developed is best described as a statistical signal-based strategy. As I try to find a metric that shows statistical evidence on which I can capitalized on. In financial markets one can profit by betting on a volatility change³ or directional bias. Capitalizing from directional bias is as simple as buying when your directional bias is positive and selling when there is a negative directional bias. (I am referring to positive and negative returns)

³a measure of how much and how quickly the price of a trading instrument fluctuates over time

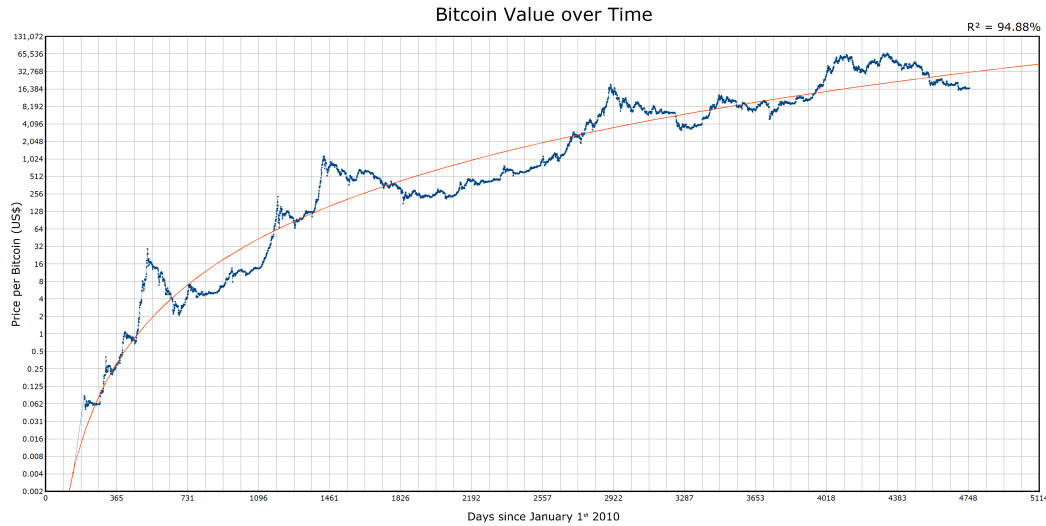


Figure 1: Bitcoin price chart

2.2 What is Bitcoin?

As my algorithm trades the Bitcoin/USD pair, this section outlines what Bitcoin is. First it is important to understand some key aspects of fiat currency systems ⁴. One of the characteristic of fiat systems is their centralized nature: governments and central banks manage the money supply through monetary policy ⁵. This gives them the ability to expand or contract the money supply, which can be used to address economic challenges but can also lead to inflation when supply increases significantly. While this system provides tools for economic management, it also requires trust in institutions to balance competing economic priorities effectively.

Bitcoin, in contrast, is a decentralized digital currency created in 2009 by an anonymous entity known as Satoshi Nakamoto.

In the figure 1 one can see the price of Bitcoin over the years. The Y-axis is logarithmic to better visualize the strong increase in price. The X-axis shows the days since 2010.

⁴Fiat money is a currency that is not backed by a physical commodity, such as gold or silver, but is instead issued by a government.

⁵actions undertaken by a nation's central bank to manage the money supply and credit conditions to achieve macroeconomic goals

Why trade Bitcoin?

I want to clarify that Bitcoin was selected as the underlying asset for my trading strategy out of practicality. While Bitcoin's fundamentals were studied through different books and personally see value in its properties as a decentralized asset, these views are irrelevant to this thesis. The trading algorithm I developed is designed solely to respond to market inefficiency signals, which can be found in any tradable asset, regardless of its fundamental characteristics.

Data accessibility

In the crypto market, real time and historical data is mostly free and accessible freely through public APIs. An API is a tool that allows programs to request and receive specific data from external services in this case, used to fetch real-time Bitcoin price and order book data from the Coinbase crypto exchange. The data I based my strategy on is totally free I just had to create my own time series in the traditional market I would have paid thousands of dollars for the same data.

Modern API infrastructure

Some crypto exchanges offer modern API infrastructure which allow to automate trading. The exchange Hyperliquid offers a world-class Python implementation of their API. This allows me to carry out trades with a simple self-written Python script.

Entry barriers

You can start trading crypto with as little as 10 USD no matter if the exchanges offers automated trading or just manual executed trades. While in the traditional market you need at least 10'000 USD to open an account for on a broker which offers automated strategy access.

These features make Bitcoin an interesting asset, both from a technological and financial perspective.

2.3 Efficient markets

Making consistent profits in financial markets is difficult. Prices move in ways that often seem unpredictable, and most traders both discretionary and systematic struggle to gain a lasting edge. One explanation for this is the Efficient Market Hypothesis (EMH), which suggests that asset prices already reflect all available information, leaving little room for profitable opportunities (Fama, 1970).

If the EMH were fully accurate in practice, then even the most advanced trading algorithms and best discretionary traders would not be able to consistently beat the market.

⁶ Yet markets are not perfectly efficient, being heavily influenced by emotions, fear, and herd behaviour. This idea is partly based on my own reflections and is also described in Dalton (2007), who emphasizes in the preface ('Markets are rational, people are not') that markets themselves may be efficient, but human behaviour often prevents rational decision-making.

Market efficiency can vary significantly across different trading markets. Less established or regulated markets often are less efficient because of lower liquidity and fewer institutional participants. One example would be that in when Bitcoin was not as big as today. Crypto Exchanges had different prices for Bitcoin which would offer an arbitrage opportunity between exchange But still some of the most valuable assets such as stocks, gold and Bitcoin prices are a reflection of human emotion pricing in information. Especially in the last few years with the introduction of spot Bitcoin ETFs a lot of volatility came in with big opportunities.

Depending on the timeframe there are still a lot of inefficiencies from which individuals can make a profit from. The strategy focuses on short-term inefficiencies with my trading algorithm because I only have about one month for the live test and it would not make sense to run an algorithm which holds a trade over weeks and takes a trade every two months. In addition The initial concept for this thesis was already based on short term signals.

Understanding Market Edge

Before diving into strategy development, it's important to understand the concept of a market edge. An edge is a consistent advantage over the market that allows a trader to capitalize on market inefficiencies. Edge can have many different origins some examples are:

- Superior information or analysis
- Faster execution or better technology

⁶Beating the market means achieving a higher ROI (Return on Investment) than a certain benchmark. A common benchmark is the S&P 500, which is an index including the 500 largest companies listed on U.S. stock exchanges. The S&P 500 has had an average yearly return of about 10.33% since 1957

- Unique trading approaches that exploit market inefficiencies
- Better risk management techniques

Without a clear edge, even the most sophisticated trading system will struggle to generate consistent profits. The goal of systematic strategy development is to identify, validate, and exploit such edges repeatably.

2.4 Development of a Systematic Strategy

Idea Generation: The systematic strategy has to be based on a hypothesis that once signal X occurs the market is wrong and we can profit from the market correcting back to a fair value. This can be done by copying other strategies and applying them in a different way or in a different market. The approach I took was trying to make a discretionary strategy into a systematic one..

Backtesting: After coming up with an idea we have to make a strict set of entry and exit conditions. We test the rules on historical data. This allows the developer to evaluate how the strategy would have performed in the past. This allows us to get a hint on how the strategy would perform in the future and if it is worth automating.

Optimization and Robustness Testing: After initial tests, parameters can be adjusted. A simple example would be adjusting the holding time of a trade. Some inefficiencies can be corrected in minutes others might take a day or two it is up to us to find the right balance. Holding time is a simple example which doesn't bring a lot of risk with it if we test on different holding times. As long as we use a limited amount and have clear differences between the different holding times we can be sure that the strategy is robust. The biggest risk we face with backtesting is that we might overfit the strategy to the past data. An overfitted strategy looks great on historical data but will fail in the live market.

Live Execution: If the backtesting results are promising and the strategy appears robust, it can be deployed live. Execution is handled by an algorithm running on a server, which listens to market data and acts as soon as the conditions are met. New things have to be taken in consideration here which are not a problem in backtests. One such thing is the time it takes to calculate the entry and exit conditions. This is a problem, as the price might change between when the actual entry conditions occur and when the entry signal is calculated.

Throughout this process, the focus remains on consistency and measurable results. Unlike human decision making, a systematic strategy must behave identically entry and exit conditions. This makes it possible to evaluate, improve and automate trading in a transparent way.

2.5 Order book

In financial markets, an order book is a real-time list of all pending buy and sell orders for a specific asset. It shows the quantity of the asset that traders are willing to buy or sell at different price levels. Understanding the order book is crucial as it provides insight into market supply and demand.

The order book consists of two sides:

- **Bid side:** Contains all buy orders, showing prices traders are willing to pay
- **Ask side:** Contains all sell orders, showing prices at which traders are willing to sell

Market participants can be classified into two main categories based on how they interact with the order book:

Passive Market Participants

These traders place limit orders that sit in the order book waiting to be filled. They provide liquidity to the market by being willing to buy or sell at specific prices. For example:

- A trader placing a buy order at \$40,000 when Bitcoin is trading at \$41,000
- A trader placing a sell order at \$42,000 when Bitcoin is trading at \$41,000

Passive traders typically earn the bid-ask spread but must wait for their orders to be filled.

Active Market Participants

These traders place market orders that immediately execute against existing orders in the order book. They remove liquidity from the market by "taking" the best available prices. For example:

- A trader immediately buying at the current ask price
- A trader immediately selling at the current bid price

Active traders pay the bid-ask spread but get immediate execution.

The strategy search process uses order book data to identify potential trading opportunities based on the imbalance between buy and sell orders.

(Coinbase; Capital.com)

3 Methodology

3.1 Project Overview

The aim of this project was to design, test, and deploy a fully automated trading strategy for Bitcoin. The development process consisted of four main stages. First, historical price data was collected to backtest strategies on. In the second stage An edge was developed by analyzing the collected data to define a signal. In the third step, the entry and exit conditions are backtested on several months of historical data using a custom Python-written framework. Finally, if the results met predefined criteria (e.g. acceptable drawdown and stable returns), the strategy was deployed live on a PaaS that connects to the exchanges where the trades are executed in real time.

Use of AI Tools During the writing process, ChatGPT (OpenAI, GPT-5, <https://chat.openai.com>) was used as a support tool to correct spelling mistakes and language refinement. All content generated with AI assistance was critically reviewed, edited, and integrated by the author. The core research, analysis, methodology, and conclusions represent the author’s original work.

3.2 Services used

Now before diving into the development of a systematic trading strategy I want to explain what kind of services we need and how we can use them to our advantage.

Broker

In order to buy and sell Bitcoin we need an exchange to trade on and the exchange should offer a modern API infrastructure, which allows the trading algorithm to send commands to the exchange to sell and buy Bitcoin. The most fitting exchange for this is Hyperliquid, as it offers a modern API infrastructure and comprehensive documentation.

Hosting services

Running the algorithm on my personal computer is not an option as it would require me to have my computer running 24/7. This is not feasible and would also require me to have a stable internet connection. For this reason a hosting service was required to run the algorithm 24/7. Railway was selected since it is very simple to use and I already know how to host my code there as it had been used previously for other projects. Railway is a PaaS (Platform as a Service) which allows me to host my code on a server and run it remotely.

Application Programming Interface

An Application Programming Interface mostly referred to as API is a tool that allows a program to request and receive a specific data form. In my use this will enable me to create my own time series dataset and open and close positions on the exchange I chose.

Coinbase offers a public API which provides market data for all their listed assets.

Hyperliquid offers a Python SDK. An SDK is a toolkit that wraps the API with code, documentation and examples.

Backtesting Python library

A Python library is a collection of tools and libraries that allow developers to interact with a specific API. For backtesting I chose vectorbt as it offers a comprehensive set of tools for backtesting with extensive documentation.

3.3 Data collection

A custom dataset was created because it offers a key advantage: it allows me to experiment with ideas and patterns that are less likely to have been explored before as I am not limited to the data provided by a platform. This increases the chance of discovering something new that might be missed when using more commonly available datasets. The consequence is that a lot of things have to be implemented through a custom implementation which costs a lot of time.

A Python script was created which sends requests to the API in one-minute intervals and stores the retrieved data inside a Postgres database. The data mining script has been running since March 11, 2025.

The script calculates the order book deltas with different depths including 1%, 2.5%, and 5% depth. The order book delta is calculated by summing up all bid or sell orders between the current price and the price $n\%$ below or above the current price and then subtracting the total value of the bid orders by the total value of the ask orders. The following formula shows this:

$$\Delta_{OB} = \sum_{i=1}^n V_{bid_i} - \sum_{i=1}^n V_{ask_i} \quad (1)$$

where:

- Δ_{OB} is the order book delta.
- V_{bid_i} is the volume of the bid order at price level i .
- V_{ask_i} is the volume of the ask order at price level i .
- n is the number of price levels included in the calculation.

After the order book deltas and the current Bitcoin price have been retrieved and calculated we add a timestamp value as well with the help of the time Python library. Then the data was added to the PostgreSQL database.⁷

⁷A live data mining progress visualization was made available during the project development phase.

Time series creation challenges

The first difficulty is to find an API endpoint that offers a full order book snapshot as most exchanges only offer the 1000 closest limit orders to the current price which is not enough to calculate the order book delta of $\Delta_5\%$ (1). The API provider also should have influence on the market so it should be an exchange with large amounts of volume. Coinbase is a good choice because it is the largest regulated crypto exchange in the United States and through Coinbase prime they provide custody, execution, and compliance tools that big players (hedge funds, corporates, ETFs) need.

#	Exchange	Trading volume(24h)	Avg. Liquidity	Weekly Visits	# Markets	# Coins	Fiat Supported
1	 Binance	\$11,549,460,205	932	11,470,617	2076	552	ARS, AUD, BRL and +86 more
2	 Bybit	\$2,180,782,777	671	4,004,450	1243	727	BRL, CHF, CZK and +72 more
3	 Coinbase Exchange	\$1,427,753,050	759	43,196	455	315	USD, AED, ARS and +61 more

Figure 2: Biggest exchanges by spot volume

The main challenge lies in creating an accurate time series as maintaining precise timing between data collection and processing is crucial. In traditional financial data, price points are recorded exactly at the start of each minute (e.g., 12:01:00, 12:02:00). Python executes code by running it line by line in a single threaded environment. Since I have a function for every order book delta depth it took about seven seconds to calculate all the order book deltas. By using a multithreaded approach I was able to reduce the time to about one second.

```
timestamp      , delta1      , delta2_5      , delta5      , ema      , price
2025-03-11 23:37:01, -1.7860111599998731, -29.81752685999936, -51.35012206999994, 81722.2992269564, 82681.515
2025-03-11 23:38:01, -9.159897549999613, -28.886295949999635, -59.16455162000017, 81724.15412891719, 82658.855
2025-03-11 23:39:01, 0.9635912400002269, -17.59676415999985, -48.747833100000264, 81727.89138381915, 82758.125
2025-03-11 23:40:02, -31.762112719999976, -37.069179229999804, -48.97528278000016, 81730.2286387211, 82838.605
2025-03-11 23:41:01, 1.5214257500002475, -15.89020499999981, -46.92668486000042, 81725.69138381915, 82742.585
2025-03-11 23:42:02, -37.1962318599999, -53.92862208999938, -51.16522616000003, 81725.3423642113, 82722.1
2025-03-11 23:43:02, -18.146995449999707, -16.766174039999927, -50.14718264999999, 81728.89530538778, 82798.935
2025-03-11 23:44:02, -11.537015529999849, -43.723078309999494, -54.20299224000007, 81729.95805048582, 82831.81
2025-03-11 23:45:01, 3.8378262200003803, -9.368316929999931, -44.171431719999816, 81727.09530538777, 82760.36
2025-03-11 23:46:01, 2.941922400000152, -14.48629792999941, -45.87833383000003, 81728.2992269564, 82732.0
2025-03-11 23:47:02, 5.238047150000227, -18.90598094999956, -38.15246496000002, 81721.44824656425, 82677.395
2025-03-11 23:48:01, -46.036056839999674, -60.33061878999956, -90.25047052000003, 81724.99334460346, 82696.15
2025-03-11 23:49:02, 13.337818910000124, -11.264916459999426, -34.003491290000056, 81723.70707009366, 82686.83
```

Figure 3: database lines example

Full process flow chart

Below you can see the combined process of creating the time series.⁸

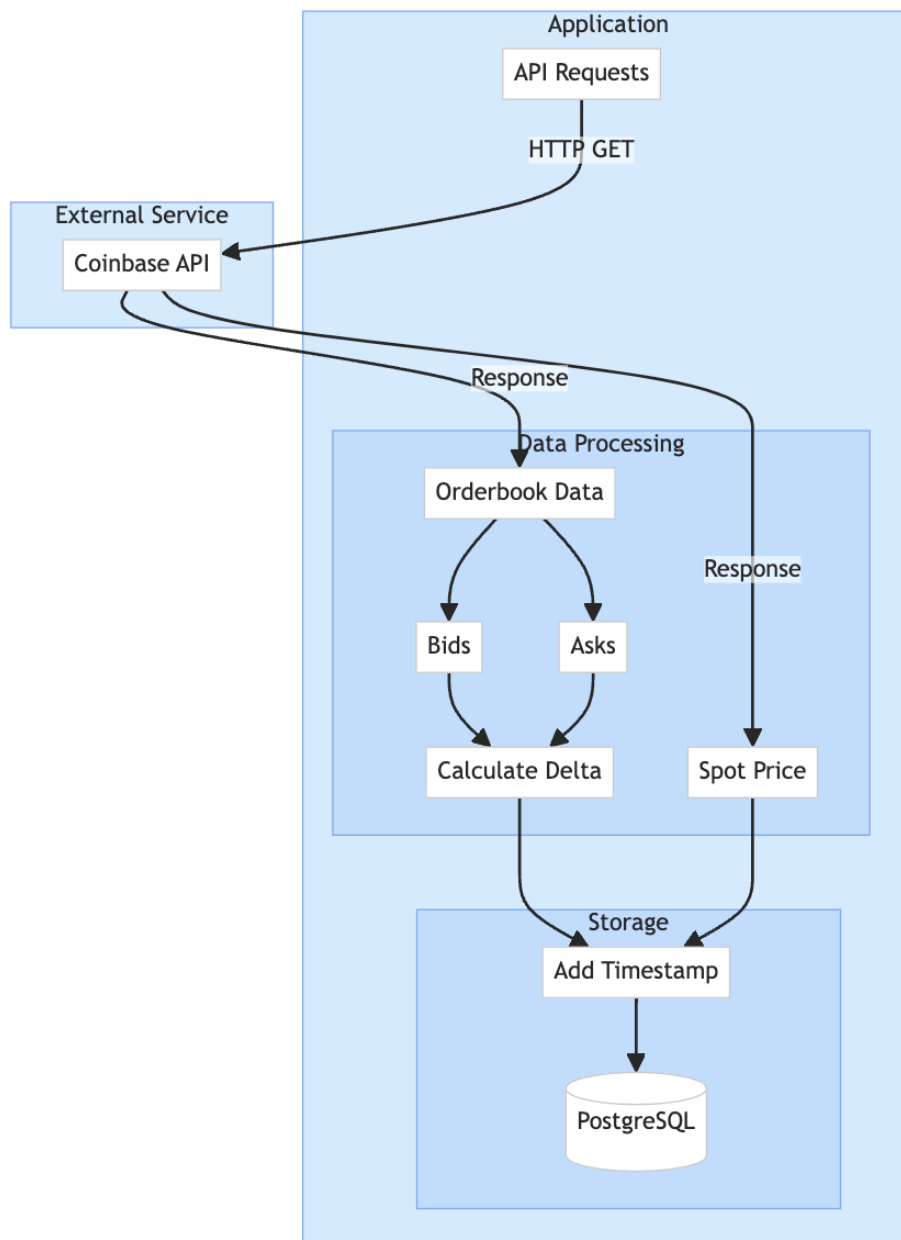


Figure 4: Time series creation progress

⁸Example datasets are available in the project repository.

3.4 Defining goals

The most difficult part of developing a trading algorithm is finding a strategy that is actually worth integrating. As explained in the Efficient Markets section, it is in principle possible to find a strategy that generates profit. The problem is that finding a strategy which consistently makes money and outperforms the market is still extremely difficult.

Before I start searching, I need to define what I am searching for. To do this, I use the S.M.A.R.T. goal framework (Specific, Measurable, Attainable, Relevant, and Time-bound), as described in the book *Building Winning Algorithmic Trading Systems* by Davey (2014). Table 1 outlines the goals for this project.

Goals

Table 1: SMART Goals for Strategy Development

Criterion	Goal	Explanation
Specific	Strategy trades Bitcoin/USD pair	Asset of choice
Measurable	Target Sharpe ratio between 1 and 2	Performance can be compared objectively
Attainable	Approx. 20 trades per month	Fits within time and data constraints
Relevant	Focus on stability and consistency, not outperforming the market	Aligns with project objectives
Time-bound	Two months to find a strategy	Aligns with thesis timeframe

3.5 Robustness Check via Resampled Strategy Variants

Instead of running a single backtest of one strategy, we generate M variations of the same base entry condition. Each variation trades a different random subset of the original trade signals and use different holding periods. It is important to note that we could use a different parameter than the holding periods but for my strategy it made most sense.

Formally, let the base entry condition identify N candidate trades. The best case scenario is if the return increases as the sample size increases. For each variation $v \in \{1, \dots, M\}$ we sample a fraction $p_v \in (0, 1]$ of these N signals (without replacement) and assign a holding period H_v . The net average return of each variation is computed after applying transaction costs c that include both fees and slippage⁹.

The key insight is that if the entry condition contains no edge, the distribution of net returns across the M variations will concentrate around zero or negative values once costs

⁹Slippage is the difference between the expected price and the actual price of a trade due to market conditions.

are applied. In contrast, if a genuine edge is present, the majority of variations will show consistently positive outcomes even though each trades a different subset of signals. We gain a better insight into our strategy signals by comparing the difference in performance between different fraction sizes. The importance of this step is explained in the strategy logic section.

One might ask why not simply reverse the signal. Let $\bar{r}$ be the average average return of the original signal. A reversed signal would only be profitable if $|\bar{r}| > c$ and $\text{sign}(\bar{r}) < 0$ often enough. In practice this condition rarely holds, since most weak or noisy signals satisfy $|\bar{r}| \leq c$, making reversal unprofitable after costs. Just to showcase this I will show a strategy which took the same trade logic as the one I am going to explain but reversed the signal.

Walk-forward testing

Even by using this multi parameter testing framework the risk of finding a good result by luck is still high. In order to prevent to being lucky with a good looking parameter combination I run the same strategy on new data. This way I can see if the strategy is still profitable and if it is I can be sure that it is not just a lucky hit. First test is done on 40'000 minutes of data if the strategy is profitable I run it on 60'000 minutes of data. If the strategy is still profitable I run it on 200'000 minutes of data. 95% of the time the strategy goes from a good looking strategy to a bad one.

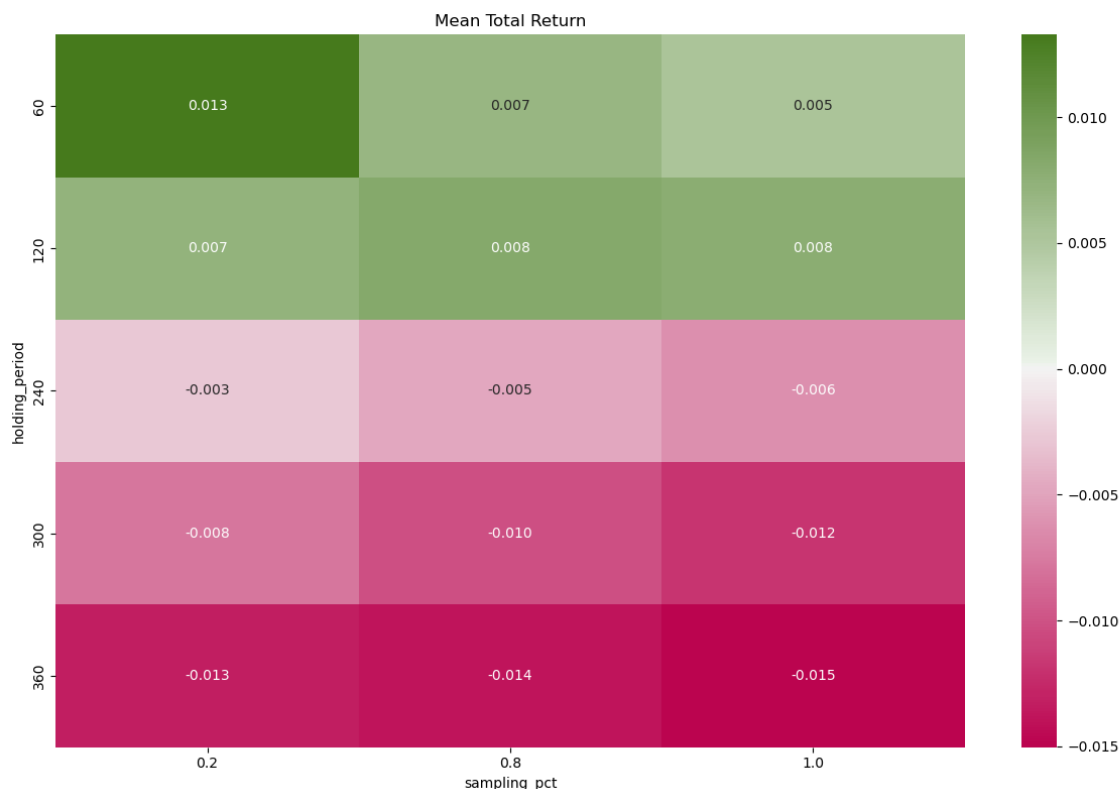


Figure 5: Backtest average return comparison

In the heatmap above 5 you can see the results for a simple strategy backtest on a 40'000 minutes time series. Strategy logic does not matter yet. The x-axis shows the sample size and the y-axis shows the average return over the different test runs. A value is colored green if the average return is positive and red if it is negative. The stronger the color the higher or lower the average return is.

A compensated version of this backtesting framework was released as the `signal_search_strategy` project. The testing and analysis methodology presented here mirrors what I use in my actual trading operations. While my original codebase contains complex trading signals and closely interlinked components, I have developed a simplified version to carrying the same logic and concepts. This implementation maintains all essential components of the system, including signal processing, cost calculations, and performance measurements, while removing private and project-specific elements. I created this streamlined version to ensure others can both verify the methodology and run the system themselves, without getting overwhelmed by technical complexities or encountering private trading information.¹⁰

¹⁰The public repository is optimized for integration with other projects.

3.6 Strategy example

As an example, a simple strategy was backtested which is based on two different moving averages. We buy once the short moving average crosses the long moving average from below and sell once the short moving average crosses the long moving average from above.

Strategy logic

$MA_{short} = 10$ (average of last 10 daily closing prices)

$MA_{long} = 50$ (average of last 50 daily closing prices)

Entry conditions: MA_{short} crosses over MA_{long}

Exit conditions: MA_{short} crosses below MA_{long}

Backtest Results

Testing period: 2014-09-17 to 2025-09-02 (4003 days)

Key performance metrics:

- Total Return: 42,116%
- Benchmark Return: 24,053% ()
- Maximum Drawdown: 72.19%
- Win Rate: 52.17%
- Total Trades: 46

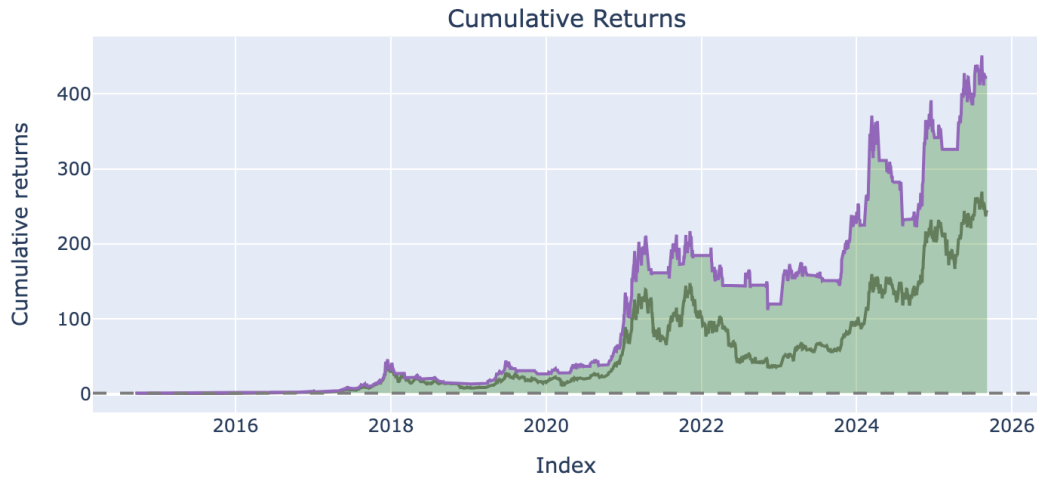


Figure 6: Equity curve of MA cross over strategy

This example is not meant as a tradable strategy. The large returns shown are mostly a result of Bitcoin's extreme growth during the testing period rather than the strategy logic itself. The maximum drawdown of 72% makes it unrealistic for real trading. The purpose of this example is to demonstrate how the vectorbt backtesting framework evaluates strategies and reports key metrics.

3.7 My strategy

The order book delta (1) was first encountered during discretionary trading. After applying the indicator for a period, it was hypothesized that it could be used as the basis for an automated strategy.

From Theory to Practice

The journey from theoretical order book delta concepts to a practical trading system involved several key steps:

1. **Initial Observation:** Through discretionary trading, it was observed that significant order book imbalances often preceded price movements, particularly in high volatility periods.
2. **Hypothesis Formation:** Based on these observations following hypothesis was formulated:
 - When large order book imbalances occur in the direction of the current trend
 - And these imbalances represent significant outliers from normal conditions
 - Then a short-term price movement in that direction is more likely
3. **Quantification Process:** To transform this hypothesis into a testable rule set following steps have to be taken:
 - Define precise mathematical measures for order book imbalances
 - Establish clear thresholds for what constitutes a "significant" imbalance
 - Create objective criteria for trend identification
 - Develop specific entry and exit rules
4. **System Development:**
 - Creation of a data collection framework for order book metrics
 - Development of outlier detection algorithms
 - Implementation of trend identification logic
 - Integration with risk management rules

Challenges in Implementation

Converting theoretical insights into practical trading rules presented several challenges:

- **Signal Noise:** While order book imbalances theoretically indicate potential price movements, many signals proved to be false positives

- **Timing Precision:** The theoretical relationship between order book imbalances and price movements needed precise timing rules for practical implementation
- **Risk Management:** Converting theoretical edge into consistent returns required careful position sizing and risk controls



Figure 7: order book delta visualization showing price action (top) and corresponding delta values (bottom)

As shown in Figure 7, the order book delta can provide valuable context at prices in high volatility regimes ¹¹. The chart displays two key components:

- The price action (top chart) shows Bitcoin price movements
- The order book delta (bottom chart) represents the imbalance between passive buy and sell orders

A high order book delta value means that there are a lot of passive buy orders below the current price. This can create a strong support level, as sellers would need strong active selling pressure to break through these orders. Even if sellers manage to push through, the accumulated buy orders can generate a correction. Same thing can happen vice versa.

¹¹High volatility regimes are characterized by large price fluctuations and rapid changes in price direction.

Order book delta outlier identification

Before applying statistical methods to identify outliers, the data was first visualized using *matplotlib*, a Python library for data plotting. This was initially done on a small sample of 7,000 data points, while the statistical tests in the final strategy use larger datasets. Several approaches were explored, but only the final method is presented here due to space limitations.

The interquartile range (IQR) is defined as the difference between the third quartile (Q_3) and the first quartile (Q_1):

$$IQR = Q_3 - Q_1$$

An outlier is defined as a value that lies outside the IQR multiplied by a factor z . In this project, $z = 2$. Outliers are then divided into two groups: red marks values below the IQR range, while green marks values above it.

When the order book delta is positive, it indicates more passive buy orders than passive sell orders. Such conditions can arise in two main scenarios: (1) price falls into a demand area where traders expect strong support, or (2) price rises and breaks through a stacked zone of passive sell orders.

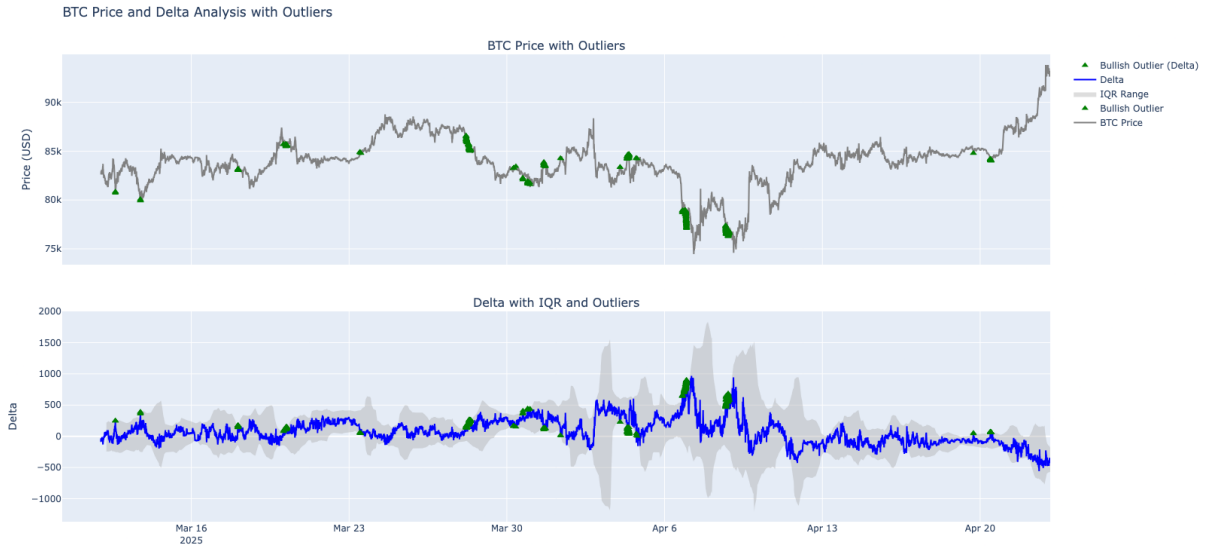


Figure 8: Outlier identification

In the upper part of figure 8 you can see the Bitcoin price plotted with outlier marked. Below you can see the order book Δ_5 plotted with the outliers marked and the IQR range marked with a grey blur. As my strategy only acts on outliers which are above the IQR range only those are marked inside of the figure.

Market Structure and Trend Identification

Bitcoin is a non-stationary asset, meaning its price is constantly changing and not predictable over a short timeframe. While long term trends can be simple to identify, short timeframes, such as on 1 minute interval data, contain substantial noise, making true trends hard to distinguish from temporary price movements.

To overcome this challenge, our approach involves aggregating the noisy 1-minute price data into a longer timeframe. Specifically, the data is converted into 240-minute OHLC¹² data. This process smooths out short-term volatility and provides a clearer view of the underlying market direction, which is essential for our trend analysis. By processing and analyzing multiple timeframes simultaneously, we can more effectively identify meaningful trends from the vast amount of market data.

¹²OHLC stands for Open, High, Low, and Close. It is a type of data that is used to represent the price action of an asset over a period of time and is also known as a candlestick chart.

Trend Analysis Overview The final trend signal is determined in a two-stage process. First, a baseline trend is identified using recent price structure. Second, the strength of that trend is assessed by analyzing the slopes of optimized support and resistance lines. The analysis is performed on 240-minute price bars.

Stage 1: Traditional Trend Analysis The initial trend label is found by looking for a pattern of higher highs and higher lows or lower highs and lower lows. An **Uptrend** requires a pattern of successively higher highs and higher lows over recent lookback windows, while a **Downtrend** requires successively lower highs and lower lows. If neither pattern is present, the trend is labeled **Ranging**.

If the trend is Ranging, an EMA ¹³ confirmation is applied. The trend is updated to match the previous trend’s direction (e.g., to Uptrend) only if the current price is also on the correct side of a long-term EMA (e.g., price > EMA).

Stage 2: Trendline Strength Analysis While Stage 1 identifies a potential trend, Stage 2 tries to identify the trend strength. This is done by fitting robust support and resistance lines over a lookback window to see if they confirm the trend’s direction. This process intentionally avoids standard regression, which can be manipulated by noise which is higher on a lower timeframe (Trading Dude, 2025). Instead it uses an optimization method to define true price boundaries. The process is as follows:

1. **Identify Pivots:** First, a preliminary, simple line-of-best-fit is calculated for the lookback window. The algorithm then finds the single high point that is furthest above this line, and the single low point that is furthest below it. These points represent the most significant moments of price rejection and are selected to serve as the definitive **anchors** for the resistance and support lines, respectively. This ensures the subsequent analysis is based on the most meaningful price action in the window.
2. **Optimize Slopes:** With the lines anchored to their pivots, the algorithm begins an iterative search for the optimal slope for both the support line ($b^{(S)}$) and the resistance line ($b^{(R)}$). The goal is to find the tightest possible fit that respects the price action, meaning the support line must never cross *above* any low, and the resistance line must never cross *below* any high. This creates a true channel around the price, and the slopes of these boundaries give a much more reliable indication of the trend’s stability than a simple average.

Based on the final, optimized slopes, a strength label is assigned. The mathematical symbols used are defined as follows:

¹³With EMA an exponential moving average with a length of 1440 based on 1 minute data is meant

$b^{(S)}$: The final, optimized slope of the support line. A positive value indicates a rising support level.

$b^{(R)}$: The final, optimized slope of the resistance line. A positive value indicates a rising resistance level.

S_t : The resulting trend strength label for the current time period.

The strength is classified according to the alignment of these slopes:

$$S_t = \begin{cases} \text{Strong Uptrend,} & \text{if } b^{(S)} > 0 \text{ and } b^{(R)} > 0 \\ \text{Strong Downtrend,} & \text{if } b^{(S)} < 0 \text{ and } b^{(R)} < 0 \\ \text{Mixed,} & \text{otherwise.} \end{cases}$$

Final Consensus Trend The final trend gives priority to strong, confirmed trends from Stage 2. If the trend is not identified as strong, it falls back to the baseline trend from Stage 1.

$$\text{FinalTrend}_t = \begin{cases} \text{Uptrend,} & \text{if } S_t = \text{Strong Uptrend} \\ \text{Downtrend,} & \text{if } S_t = \text{Strong Downtrend} \\ \text{Stage 1 Trend,} & \text{if } S_t = \text{Mixed} \end{cases}$$

This layered approach effectively filters market noise while capturing significant, high-timeframe directional moves.

logic behind the trendline automation

The idea of looking back and checking for swing lows and highs came up during a random coding session. The trendline automation is inspired from the open-source work provided by Neurotrader888 (Neurotrader888, 2023). While the underlying idea was taken from his Trendline Automation implementation, the code from his repository was modified to fit into the backtesting framework and trading algorithm. More information inside of A.2

Sharpe Ratio

To compare different strategies, it is not enough to look at total returns alone. Since investing always involves risk, a better approach is to measure the return generated per unit of risk taken. The Sharpe ratio is a standard metric for this and was used as the primary measure to compare strategies in this project.

The original formula as defined by William Sharpe Sharpe (1966, 1994) is:

$$\text{Sharpe Ratio} = \frac{\bar{r} - r_f}{\sigma} \quad (2)$$

where:

- $\bar{r}$ represents the average return of the strategy over a given period.
- r_f is the risk-free rate of return for the same period.
- σ is the standard deviation of the strategy's returns (a measure of volatility).

However, there is no universally accepted risk-free rate in the cryptocurrency market. Therefore, for this thesis, the risk-free rate r_f is set to 0. To make the ratio comparable across different timeframes, it is annualized. The modified formula used in this project is:

$$\text{Annualized Sharpe Ratio} = \frac{\bar{r}}{\sigma} \cdot \sqrt{N} \quad (3)$$

where:

- N is the annualization factor. This will be used if comparing strategies which are based on different timeframes.

4 Results

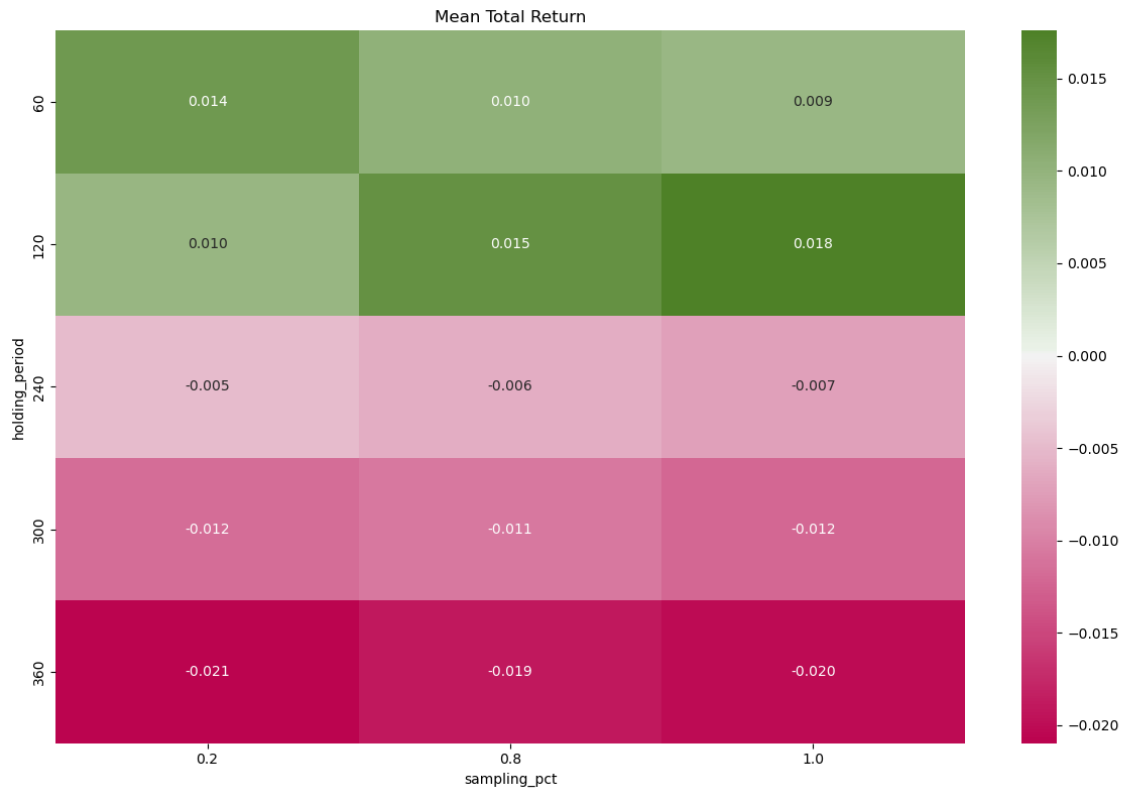
The first version of my trading algorithm was started on the 20th of July 2025. On average there is one signal per week. It took a week until signs came up that the algorithm was not working correctly. The problem was that the signal calculation was not the same as in the backtest. The backtest codebase became very large to about 5000 lines of code which made it easy to get lost. Two different logics were mixed up, causing the algorithm not to work as intended. After not being able to find the problem updates were made until similar results as in the backtest were achieved. On the 10th of August, the algorithm was able to nearly match the entry signals of the backtest. Further attempts to make the algorithm work exactly as in the backtest had to be stopped as time was running out. On the 14th of August, the first few trades were taken.

4.1 Backtest results

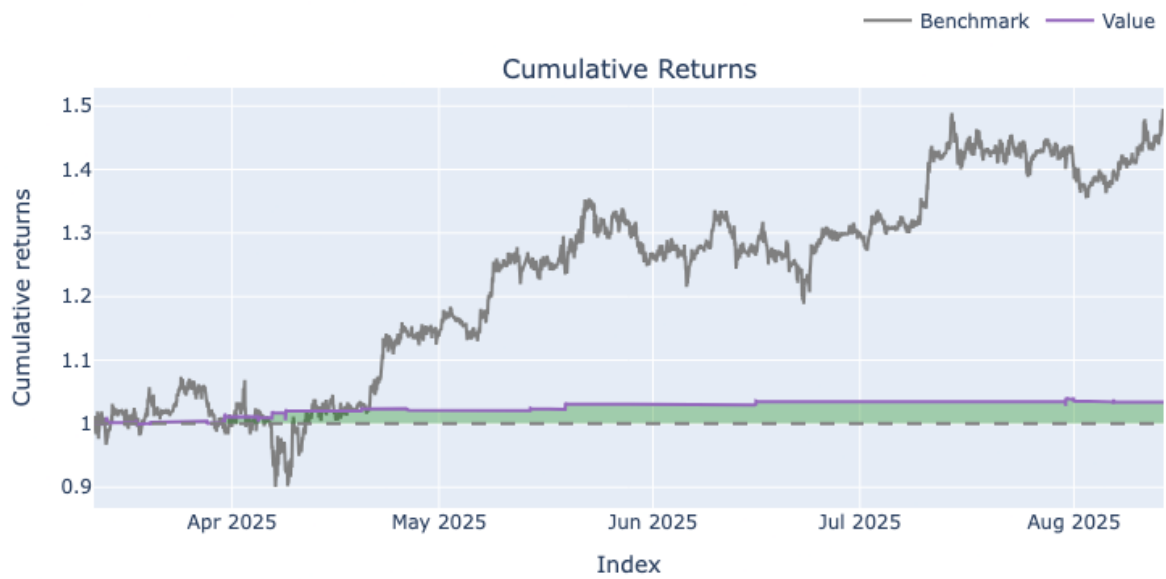
Two different sets of backtest results are presented here: results obtained before the trading algorithm was implemented, and results obtained after implementation. The pre-implementation results represent the signals based on which this particular strategy was selected.

Backtest results before implementation

Figure 9a shows the results of backtests conducted with different parameter settings. The results from this random signal combination show some interesting differences between holding periods. While both 60-minute and 120-minute horizons produced positive average returns, their behavior across sampling sizes diverged. At 60 minutes, average returns decreased as more trades were included, suggesting that profitability was driven by a smaller subset of favorable signals and may not generalize well. In contrast, the 120-minute horizon showed the opposite effect: returns increased with larger sampling sizes, indicating that the entry condition is more stable and benefits from scale. For holding periods of 240 minutes and longer, the strategy consistently generated negative results across all sample sizes. This confirms that the signal only provides an edge in the short term, and that profitability quickly decays if trades are held for too long. Taken together, these findings illustrate how robustness testing can uncover whether a strategy's apparent profitability stems from a genuine, scalable edge (as in the 120-minute case) or from overfitting to a small number of lucky trades (as in the 60-minute case). Figure 9 shows the equity curves before implementation and live trading results.



(a) Equity curves before implementation



(b) Live trading results

Figure 9: Equity curves before implementation (a) and live trading (b)

4.2 Backtest results in comparison to live trading

Table 2 shows the performance comparison between the backtest results before implementation and the live trading results.

Metric	Backtest (154 d)	Live (18 d)
Total Return (%)	+3.4	-0.0
Excess Return vs BTC (%)	-45.8	+5.6
Win Rate (%)	58.3	52.9
Sharpe Ratio	1.62	-0.007
Max Drawdown (%)	1.40	0.40
Profit Factor (ratio)	2.99	0.999
Number of Trades	24	17
Time period (days)	154.0	18.0
BTC Sharpe (benchmark)	2.64	-2.35

Table 2: **Performance Comparison:** Backtest vs. Benchmark vs. Live Trading. Live results reflect the latest run with explicit units (quote currency or percent). Excess return is strategy return minus BTC return over the same live period.

Figure 10 shows the strategy returns and Bitcoin returns over the time the strategy was actively deployed on the market. Blue line shows the strategy returns and the orange line shows the Bitcoin returns.

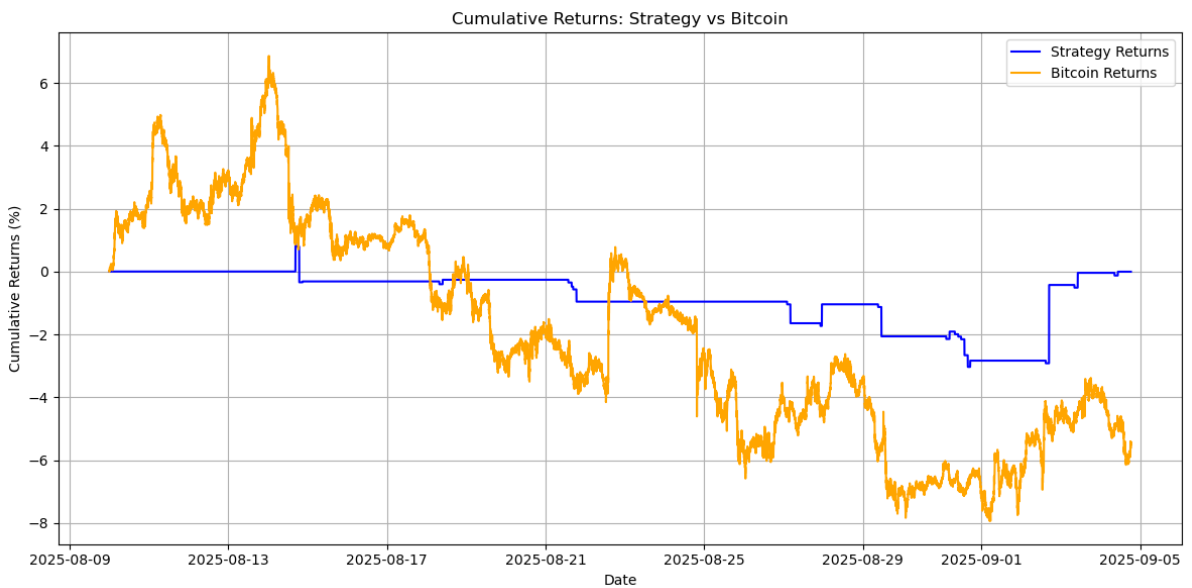


Figure 10: Performance comparison

4.3 Strategy evaluation

The live trading period was short, resulting in only 17 trades, which are not enough to draw any conclusion. A statistically significant sample size would be closer to 80-100 trades same can be said about the Backtest as it only includes 24 trades. The performance results reflect this uncertainty: the Sharpe ratio dropped from a promising 1.62 in the backtest to nearly zero (-0.007) live, and the Profit Factor fell from a strong 2.99 to a break-even level of 0.999.

This performance decline can be connected to different factors. The most significant is the limited number of trades, meaning the results could easily be due to random market fluctuations. Furthermore, there were challenges in perfectly replicating the backtest logic in the live environment. Even small differences between a backtest model and a live algorithm can lead to significant differences in performance. Given only 17 trades, these implementation differences cannot fully explain the performance gap.

While the strategy's absolute return was flat, its performance compared to Bitcoin was positive (+5.59%). This indicates that during this specific live period, the strategy gave better risk-adjusted performance than a simple buy and hold strategy. So, while it did not generate profit, it successfully beat the market with less drawdown than the underlying asset

Another notable factor was the difference in market regime. During the live trading period Bitcoin was in a downtrend including strong price drops, in contrast the backtest covered periods with more short term uptrends. Since the strategy is designed to buy strong pullbacks within uptrends, this explains why more entry signals occurred in the backtest than in live trading.



Figure 11: Live execution of trading algorithm on Hyperliquid

5 Discussion

5.1 Interpretation of results

The central question of this thesis was whether it would be possible for an individual to develop an algorithm that achieves positive returns or beats a buy and hold strategy for Bitcoin. Although the algorithm did not generate absolute profits, it outperformed Bitcoin by +5.59% during the live test period. This indicates that under certain market conditions, an individual developed algorithm can temporarily outperform buy and hold.

These results can be interpreted through the lens of the Efficient Market Hypothesis (EMH) 2.3. The difficulty in achieving large, consistent profits aligns with the EMH's assertion that markets are generally efficient. However, the algorithm's ability to outperform the benchmark during a market downtrend suggests it successfully identified and capitalized on a specific, short term inefficiency. This supports the idea that while markets may be efficient on average, exploitable inefficiencies can still exist within certain market regimes, which is where a systematic strategy can provide value.

At the same time the number of trades was limited (17 during live trading and 24 during the backtest). (Davey, 2014) suggests a sample size of 30-100 trades during live trading is considered to be a good sample size. From a statistical seen perspective, 17 trades cannot be considered robust evidence of a persistent edge. The observed outperformance should therefore be seen as a first indication that this strategy has the potential to be profitable and not as a definitive proof.

5.2 Challenges

Writing the full thesis in LaTeX was a valuable opportunity to further develop my skills. However, it also led to time-consuming troubleshooting due to technical aspects that still needed to be learned. The most unexpected challenge was transitioning from backtesting to live implementation. While the backtesting framework performed well, the live environment required a completely different structure, as all processes had to run in real time.

The strategy searching was the part which took most of the time and effort. It was the only part of this project where there was not a clear goal. The learning which can be taken from this this process is how much a good framework is needed to make the process of testing and evaluating ideas easy and effectively. Entering every parameter entry condition into a single vectorbt backtest is very tedious. The first version of the backtesting framework when ran crashed my computer in a matter of five minutes as it was not optimized to run on computers with lower RAM. The parameter search engine runs 1000 strategies at the same time in order to do this on computer with less ram the different strategies were split into batches of 100 and ran in a sequentially manner. This

change made it possible to use but came with the compensation of taking more time.

5.3 Limitations

The scope of this project was ambitious for a Matura Thesis. The data collection began in March 2025, which limited the total amount of historical data available for backtesting and model validation. A longer data history would have allowed for more robust out of sample testing across different market cycles. Due to these time constraints, certain areas, such as better optimization of the strategy parameters or the implementation of regime based activation of the strategy, could not be explored in this project.

5.4 Practical insights

Bitcoin is very efficient, and smaller individual traders have to play in areas where they can make use of their advantage they have over big institutions. Big institutions are weighed down by mandates, regulation, scale, and career risk. On an asset like Bitcoin most of these advantages do not come to play. A US regulated algorithm trading firm might not have access to coins which are only tradeable on brokerages not in the US which comes down to less big players in the market making it less efficient. These limitations may give smaller traders a chance to find short-term opportunities, especially in markets like Bitcoin where big institutions are less active.

5.5 Future work

This project serves as a strong foundation for several future research directions. The following are potential next steps to build upon this work:

- **Extended Live Trading Period:** The most important next step is to continue running the algorithm to gain a statistically significant number of trades (ideally 100+). This would provide a much clearer picture of its long term robustness and performance.
- **Application to Alternative Assets:** As discussed, Bitcoin is a highly efficient market. Applying the same order book delta logic to smaller, less liquid altcoins could bring a stronger edge and potentially better performance.
- **Portfolio Integration:** This strategy was designed to buy strong pullbacks in a short term uptrend. The broader market regime still has to be investigated. If identify it could be integrated into a multi strategy portfolio, where other algorithms are designed to trade different market regimes (e.g., momentum in a downtrend, or range-bound trading). This would create a more robust and higher Sharpe ratio portfolio.

5.6 Final thoughts

As this project comes to an end, I realize how much effort I have put into it. When I started this thesis, I wanted to do everything required to build a profitable trading system no matter what it would take. This project became my main focus for months and my main activity during the summer vacation. I invested not only time, but nearly all of my energy and curiosity into building it. Looking back, this work reflects far more than just an algorithmic trading project — it shows my persistence, the skills I have developed, and how deeply I engaged with the subject.

References

- Saifedean Ammous. *The Bitcoin Standard: The Decentralized Alternative to Central Banking*. Wiley, 2018.
- Capital.com. Passive order definition. <https://capital.com/en-int/learn/glossary/passive-order-definition>. Accessed: September 6, 2025.
- Coinbase. What is an order book? <https://www.coinbase.com/en-gb/learn/advanced-trading/what-is-an-order-book>. Accessed: September 6, 2025.
- James Dalton. *Markets in Profile: Profiting from the Auction Process*. Wiley, Hoboken, NJ, 2007.
- Kevin J. Davey. *Building Winning Algorithmic Trading Systems: A Trader's Journey From Data Mining to Monte Carlo Simulation to Live Trading*. Wiley, 2014.
- Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- Neurotrader888. Technical analysis automation - trendline automation. https://github.com/neurotrader888/TechnicalAnalysisAutomation/blob/da99c20bf3d977b639451258cd6cfca9baa1dcc3/trendline_automation.py, 2023. Accessed: September 6, 2025.
- William F. Sharpe. Mutual fund performance. *The Journal of Business*, 39(1):119–138, 1966.
- William F. Sharpe. The sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994.
- Trading Dude. Trend reliability vs randomness across different timeframes. <https://medium.com/@trading.dude/trend-reliability-vs-randomness-across-different-timeframes-be3a79f5ea7f>, 2025. Accessed: September 6, 2025.

A Appendix

A.1 List of figures

Figure 1: Bitcoin price chart (data source: Reddit visualization)

Figure 2: Biggest exchanges by spot volume (data source: CoinMarketCap)

Figure 3: Author-generated screenshot from database CSV file

Figure 4: Created by explaining my code to claude-3-5-sonnet

Figure 5: Author-generated plot showing outlier comparison (code: GitHub repository)

Figure 6: Moving average crossover strategy visualization (generated by vectorbt framework)

Figure 7: Order book delta visualization showing price action and delta values from Coinglass Supercharts

Figure 8: Outlier identification visualization (author-generated) using the Plotly Python library

Figure 9a: Backtest results before implementation (generated by custom framework: signal-search-strategy)

Figure 9b: Live trading results equity curve (generated by vectorbt framework)

Figure 10: Performance comparison chart (data: performance_comparison.csv)

Figure 11: Live execution screenshot from Hyperliquid platform

A.2 Code sources

Trendline Automation [6. September 2025] This was used in §3.7 My strategy (Trend Analysis Overview). Commit hash: da99c20bf3d977b639451258cd6cfca9baa1dcc3

A.3 List of tables

Table 1: SMART Goals for Strategy Development table just used for better understanding the goals of the project.

Table 2: Performance comparison between the backtest results before implementation and the live trading results [6. September 2025]. Data used:

- Backtest file
- Live trading hyperliquid
- Bitcoin benchmark for live trading

A.4 Published Github repositories

signal_search_strategy [6. September 2025]

research_notes [6. September 2025]

data_used [6. September 2025]